

Analýza PLC programu pro stanici OP10

Projekt PLC pro stanici OP10 je složitý systém řízení s mnoha modulárními bloky pro dílčí zařízení (stoppery, bezpečnostní brány, lepení, NG boxy, aj.). Jeho zdrojový kód je rozdělen do řady funkčních bloků (FB/FC), datových bloků (DB) a organizačních bloků (OB), což je podle standardu IEC 61131-3 správný přístup. Nicméně z analýzy vyplynulo několik zásadních problémů: nejednoznačné názvosloví, zastaralé („OLD SHIT“) či nepoužívané části kódu, nedostatečné komentáře a modulární dělení, rozsáhlé globální datové struktury a potenciální bezpečnostní slabiny. Níže shrnujeme klíčové nedostatky, jejich dopad a navrhuje opatření k rychlé nápravě.

Struktura kódu a modularita

- **Nekonzistentní struktura a názvosloví:** V projektu se střídají bloky pojmenované podobně (např. `Gluing.xml` vs. `GluingControl.xml`) nebo velmi obecně (např. „SafetyGate“, „Pilz“), což ztěžuje orientaci. Některé složky (např. „OLD SHIT“) obsahují zřejmě nepoužívaný kód, který zvyšuje složitost údržby. Podle zásad psaní kódu pro PLC by měl být kód **čistý a homogenní**, aby byl lépe udržovatelný ¹. Doporučujeme odstranit nebo archivovat zastaralé bloky a zavést přehledné pojmenování (např. používat smysluplné prefixy/suffixy podle funkcionality) a sjednotit strukturu složek.
- **Modularita a rozdělení funkcí:** Některé bloky jsou rozsáhlé a plní více rolí (kombinují logiku řízení i kontroly). **Modularizace** kódu do menších, specializovaných bloků usnadňuje čtení, testování i další rozvoj ². Navrhujeme rozdělit složité funkční bloky na samostatné modulární FB/FC (např. oddělit logiku IO, logiku sekvence stroje a bezpečnostní logiku do samostatných bloků).
- **Komentáře a dokumentace:** Škoda, že projekt neobsahuje srozumitelnou dokumentaci přímo v kódu (většina XML exportů postrádá podrobné komentáře). Doporučuje se doplnit komentáře k jednotlivým UDT, FB a rozhraním. Komentované kódy a jednotné **pojmenovávací konvence** zvyšují přehlednost ¹. Například definovat `UDT_GluingParameters` se všemi příbuznými proměnnými lepení nebo `FB_StopperControl` s jasně pojmenovanými vstupy/výstupy.

Funkčnost a spolehlivost

- **Chybějící validace vstupních dat:** Většina řídicích bloků pracuje s hodnotami ze senzorů, HMI nebo databází (např. procesní parametry). Program aktuálně neobsahuje dostatečné kontroly, zda jsou tyto vstupy v rozumných mezích (např. kladné hodnoty, maximální rozsahy). Bez validace hrozí **přetečení čítačů nebo podtečení**, chyby v logice (např. záporný čas) a potenciálně i bezpečnostní incident. Doporučujeme přidat **kontroly konzistence a rozumnosti vstupních proměnných**, jak radí bezpečné programování PLC – například ověřovat, že čítače a časy nejsou záporné a nekumulují mimo platný rozsah ³.
- **Bezpečnostní logika a reakce na chyby:** Kód bezpečnostních modulů (Pilz, SafetyGate) by měl obsahovat jasné stavy pro poruchové podmínky (fault), sledování časovačů (watchdog) a náhradní chování. Je potřeba zkontrolovat, že při nečekaném stavu (vypnutí energie, selhání senzoru) program bezpečně přejde do nouzového režimu. Doporučuje se zpřísnit ošetření chyb – např. přidat diagnostiku uzavření bezpečnostních smyček a přebudovat řídicí sekvence tak, aby chyby nezůstaly ignorovány.
- **Soudržnost logiky:** Z analýzy vyplynulo i několik potenciálních logických závad: například přímé ovládání výstupů bez příslušných resetů nebo nedostatečné sekvenční řízení. Při složité interakci

více zařízení je vhodné zavést transparentní **stavový stroj** (např. UDT s definovanými stavy procesu), aby se předešlo nejednoznačným nebo ohnutým podmínkám. Tím se zmírní riziko „spaghetti logiky“ a zlepší laditelnost programu ² ¹.

Výkon a efektivita

- **Velké globální databáze:** Projekt obsahuje rozsáhlé datové bloky a mnoho globálních proměnných (např. `Default variable table` ~85 kB, obsáhlé I/O mapy). Při každém cyklu PLC se prohledávají celé struktury, což může prodlužovat scan time. Doporučuje se **ořezat** nekritické proměnné, které se nevyužívají, a místo obrovských ploch `BOOL` použít *strukturální datové typy (UDT)* nebo pole. Seskupení příbuzných proměnných do UDT zjednoduší přístup a často i zmenší spotřebu paměti (typicky výhodou je lepší zarovnání bitů) ⁴ ⁵.
- **Cyklus PLC a výpočetní náročnost:** Logika řízení by měla být optimalizována pro reálný čas. Je třeba zkontrolovat, zda nejsou v cyclic routines (např. OB1) zbytečně časově náročné smyčky nebo výpočty (matematika, řetězové akce) – ty lze přesunout do časovaných rutin nebo ON/OFF logiky. Zbytečné vyhodnocování (např. často volané `MOVE` a konverze) snižuje odezvu systému. Doporučujeme **měřit scan time** při simulaci a identifikovat úzká místa (profilovat kód). Následně optimalizovat například vzájemně závislé kontroly do jednoho bloku místo opakování.
- **Udržování datových záloh a logů:** Pro případ chyby nebo útoku by bylo vhodné mít schopnost rychle obnovit hodnoty (např. klíčové parametry). Doporučuje se implementovat zálohování kritických dat (externí DB nebo interní „retained“ proměnné) a logování událostí.

Bezpečnostní rizika

- **Nechráněná komunikace a zápisy:** PLC obvykle komunikuje s HMI, SCADA či nadřazenými systémy (OPC UA, Profinet). V kódu by měla být striktně omezena práva zápisu do databází a bitů. Podle **bezpečnostních doporučení** by měly být vyhrazené registry/bloky pro různé funkce – tak, aby např. externí zařízení zapisovala jen do schválených bufferů ⁶. V projektu je třeba ověřit, že HMI nebo nadřazený systém nemůže přepsat vnitřní mezivýpočty nebo cokoli mimo určených adres. Zavést validaci příchozích dat (dvoufázové zápisy do „back-up bufferů“ a teprve pak přecvaknutí do hlavních proměnných).
- **Autentizace a hardening PLC:** Zajištění bezpečnosti programu se neděje jen v kódu, ale i v konfiguraci PLC. Dle *ISA/IEC 62443* a NIST doporučení je vhodné vypnout nepoužívané porty/protokoly na PLC a omezit přístupy (např. heslem pro upload/download programu) ². V kódu to lze reflektovat přidáním ochranných mechanismů, např. zamezení neautorizovaných úprav (interní „isRunning“ proměnná) nebo detekci neoprávněné změny datového bloku.
- **Chybějící kontrola integrity a redundance:** Pro kritické vstupy (stop spínače, senzory bezpečnosti) doporučujeme *plausibilní kontroly* či redundanci (např. porovnání více senzorů) ³. I když PLC logiky nemají běžně mechanismy jako CRC, lze programově zkontrolovat konstantnost signálu (filtr časových skoků) a reagovat na podezřelé hodnoty.

Shrnutí problémů a doporučení

Problém	Dopad	Doporučení	Priorita
Nekonzistentní názvosloví a složité struktura	Snížená přehlednost, chyby při údržbě	Zrušit „OLD“ kód, sjednotit názvy (jasné funkční pojmenování), zavést komentáře ¹ ²	Vysoká (údržba)

Problém	Dopad	Doporučení	Priorita
Monolitické bloky bez dělení na dílčí funkce	Obtížné testování, pomalá úprava	Rozdělit FB/FC do menších, specializovaných bloků (modularizace) ²	Vysoká (rozsah)
Neověřené vstupní/externí hodnoty	Riziko chybného chování, přetečení	Přidat validace vstupů (rozsahy čítačů, neg. hodnoty) a plausibilní kontroly ³	Kritická (bezpeč.)
Široké globální DB, nevyužitě proměnné	Zbytečné prodloužení cyklu, vyšší paměťový nárok	Odstranit nepoužívané proměnné, používat UDT pro seskupování souvisejících dat ⁴ ⁵	Střední
Nechráněné registrační oblasti pro externí vstupy	Bezpečnostní mezery (možnost přepsání)	Využívat vyhrazené registry/ datové bloky pro zápis čtení (secure coding) ⁶	Kritická (bezpeč.)
Neúplné ošetření chybových stavů	Hrozba selhání stroje, nepředvídatelné stavy	Implementovat watchdog, bezpečnostní stavy, postupné přechody (state machine)	Vysoká (bezpeč.)
Chybějící dokumentace a standardy kódu	Ztráta know-how, složitá údržba	Zavést kodex podle IEC 61131-3 (pojmenování, komentáře, formát) ¹	Střední

Legenda priorit: Kritická – okamžitá akce (bezpečnost), Vysoká – důležitá (stabilita a údržba), Střední – vhodné provést později.

Testování a nasazení opravené verze

Plán testování by měl být vícestupňový:

- Jednotkové testy:** Pro každou funkční logiku (FB/FC) vytvořit testovací scénáře se simulovanými vstupy. Například testovat FB lepení s různými vstupními kombinacemi hodnot a ověřit požadované výstupy. Použít simulátor TIA Portal (PLCSIM) nebo externí testovací nástroje.
- Integrační test:** Otestovat spolupráci bloků (např. celý cyklus stanice OP10). Simulovat komunikaci s HMI a nadřazenými systémy. Zkontrolovat, že všechny chyby a meze jsou správně zachyceny a vyhodnoceny.
- Bezpečnostní testy:** Provést režim fail-safe (vypnutí, porucha) a ověřit, že PLC reaguje podle očekávání (např. E-stop, ztráta signálu). Testovat ochranné mechanismy na registrech (pokud jsou implementovány).
- Výkonový test:** Měřit scan-timery a ověřit, že po optimalizacích je odezva odpovídající specifikacím stroje. Zvláště sledovat čas vyřízení kritických smyček.
- Nástroje a automatizace:** Pokud je možné, integrovat projekt do systémů CI/CD (např. TIA Portal Openness pro automatické exporty, spouštění testů). Založit repozitář s verzováním kódu a dokumentace.

Nasazení: Po úspěšném testování provést nasazení nejprve do vývojového/simulovaného prostředí. Pro ostré nasazení doporučujeme: - Zálohovat původní PLC projekt a konfiguraci. - Postupně nasazovat změny (např. nejprve bezpečnostní modul, poté další části) mimo provozní dobu. - Mít připravenou

možnost návratu k předchozímu softwaru. - Monitorovat PLC po nasazení (logování alarmů a cyklů) a vyhodnocovat případné odchylky.

Závěr

Shrnuto, současný PLC kód funguje, ale jeho údržba a rozšiřování je ztíženo několika programátorskými „směry“. Doporučujeme provést postupnou refaktORIZACI podle výše uvedených zásad (modularizace, validace, pojmenování) a zavést rutinní prověřování kvality (code review, statické analýzy). Takto bude projekt spolehlivější, bezpečnější a připravený na budoucí rozšíření s minimalizovanými riziky. V souladu s doporučeními průmyslových standardů tak dosáhneme vyšší kvality kódu a bezpečnosti řídicího systému ¹ ² ⁶ .

Zdroje: Doporučené praxe vycházejí z průmyslových standardů a literatury o PLC programování a bezpečnosti (IEC 61131-3, ISA/IEC 62443). Příklady a principy jsou doloženy odbornými zdroji ¹ ² ⁶ ³ .

¹ IEC 61131-3: Coding Guidelines – Stefan Henneken

<https://stefanhenneken.net/2016/04/08/iec-61131-3-coding-guidelines-2/>

² ³ PLC Security: Boost Your Defenses with Top 20 Secure Practices

<https://www.compliance-labs.com/topic/nist-sp-800-82/plc-security-boost-your-defenses-with-top-20-secure-practices/>

⁴ UDT Basics | PLCTalk - Interactive Q & A

<https://www.plctalk.net/forums/threads/udt-basics.142368/>

⁵ Standard e Best Practice di Programmazione PLC per Codice Industriale – Industrial Monitor Direct

<https://industrialmonitordirect.com/it/blogs/knowledgebase/plc-programming-standards-and-best-practices-for-maintainable-industrial-code?srsItd=AfmBOoo8PrnzCkg5VAOCJEHNM-8sgwhB8x8GOFzWYn1HzbvDnWv-JJeE>

⁶ plc-security.com

https://plc-security.com/content/Top_20_Secure_PLC_Coding_Practices_V1.0.pdf